

Teil I

Einführung

Programmieren

Programmieren: Modellierung einer realen Aufgabenstellung in einer formalen Programmiersprache und Lösung auf dieser Ebene, so dass Modell und Lösung automatisch verarbeitet werden können

→ Verbindung von Prinzipien und „Handwerk“

Um Themen mit praktischen Beispielen illustrieren zu können, folgt eine Zusammenfassung elementarer Konzepte und Techniken.

1. Prinzipien

- Umgebung
- Programme
- Klassen und Methoden
- Variablen
- Anweisungen und Kontrollstrukturen
- Typen und Ausdrücke

2. Praxis

Umgebung

Betriebssystem

Betriebssystem (Operating System, OS) steuert den Computer, ermöglicht insbesondere:

- **einheitlichen** Zugriff auf **Hardware**
(Prozessor, Speicher, Grafik, Netzwerk, Geräte, ...)
- Verwaltung von Daten in **Dateien**
- Ausführung von Programmen als **Prozesse**

verbreitete Vertreter:

- **UNIX**-Derivate (**POSIX** als gemeinsamer Standard)
 - **GNU/Linux**-Varianten incl. **Android**
 - **Apple macOS** und **iOS**
- Varianten von **Microsoft Windows**

hier: nur **Benutzung** von Betriebssystemen

hier: nur kurze Vorstellung, da viele Einführungen existieren

Kommandozeile

OS für Anwendergeräte bieten grafische Benutzeroberfläche (Graphical User Interface, GUI): Fenster, Icons, Maus-/Touchscreen

alternativ rein textbasierte Steuerung per **Kommandozeile**:

- Terminal/Shell in UNIX-Derivaten
- Eingabeaufforderung oder PowerShell in MS Windows

für das Verständnis arbeiten wir zunächst auf dieser Ebene

Zugriff auf die Kommandozeile aus Hauptmenü durch

- *Xterm*/*Terminal*/... auf POSIX (oder *xterm* ausführen)
- *Eingabeaufforderung* bzw. *PowerShell* in MS Windows (oder *cmd* bzw. *powershell* ausführen)

Kommandos

in Kommandozeile signalisiert **Eingabeaufforderung/Prompt**

- **POSIX** z.B.: `$ % ...`
- **MS Windows** i.d.R.: `C:\...>`

dass ein **Kommando** einzugeben und nach *Enter* auszuführen ist:

<code>\$ echo Hallo</code>	← <i>Kommando</i>
<code>Hallo</code>	← <i>Ausgabe</i>
<code>\$</code>	← <i>neuer Prompt</i>

auf Kommandos (hier: `echo`) folgen ggf. **Argumente** (hier: `Hallo`)

danach kann das nächste Kommando ausgeführt werden

Benutzer führen so einen **Dialog** mit dem Betriebssystem

Dateien und Verzeichnisse

viele Kommandos betreffen die **Verwaltung von Dateien (Files)**

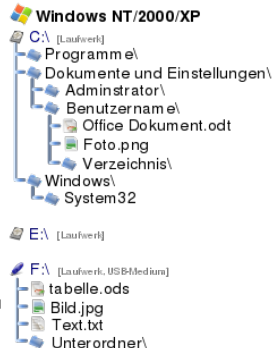
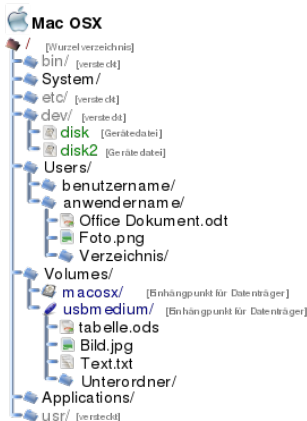
Namen von Dateien können durch **Endungen** (nach einem Punkt) die Art des Dateiinhalts beschreiben, z.B.:

<code>name.txt</code>	purere „Schreibmaschinentext“
<code>name.pdf</code>	Dokument in Druckqualität
<code>name.html</code> / <code>name.htm</code>	Hypertextseite (für Internetbrowser)
<code>name.jpg</code> / <code>name.png</code> / ...	Bild
<code>name.mp3</code> / <code>name.flac</code> / ...	Musik

ein **Verzeichnis** (**Ordner/Folder/Directory**) ist eine spezielle Datei, die andere Dateien enthält und so deren Organisation ermöglicht

Dateisystem

Verzeichnisse können Verzeichnisse enthalten, bilden so **Hierarchie**:



Quelle: Public Domain

Absoluter Pfad

absoluter Pfad führt von **Wurzel (Root)** des Dateisystems

- / auf POSIX
- Laufwerksbuchstabe + Doppelpunkt in MS Windows

durch die weiteren enthaltenden Verzeichnisse zu einer Datei

Übergänge werden dabei durch **Schrägstriche** bezeichnet:

/home/anwendername/Foto.png

← *POSIX*

C:\D.u.E.\Benutzername\Foto.png

← *MS CMD*

per Pfad greifen Kommandos auf Dateien zu, z.B. zum Löschen:

\$ rm /home/anwendername/Foto.png

← *POSIX*

C:\...> del C:\D.u.E.\Benutzername\Foto.png

← *MS CMD*

Verwendung absoluter Pfade ist recht umständlich

Relativer Pfad

Kommandozeile verwaltet ein jeweils „aktuelles“ Verzeichnis – initial das **Heimverzeichnis** (z.B. `/home/anwendername/`)

relative Pfade beginnen im aktuellen Verzeichnis, können **statt absoluten** verwendet werden

Dateien im aktuellen Verzeichnis lassen sich so direkt referenzieren:

```
$ rm Foto.png ← POSIX  
C:\D.u.E.\Benutzername> del Foto.png ← MS CMD
```

in Pfaden beschreibt

- .. das jeweils übergeordnete Verzeichnis
- . das jeweilige Verzeichnis
- das Heimatverzeichnis (nur am Anfang des Pfades)

Information zu Verzeichnissen

aktuelles Verzeichnis angeben:

<code>pwd</code>	← <i>POSIX</i>
<code>cd</code>	← <i>MS CMD</i>

Setzen von `verz` als aktuelles Verzeichnis:

<code>cd verz</code>	← <i>POSIX & MS CMD</i>
----------------------	-----------------------------

Inhalt des Verzeichnisses `verz` ausgeben:

<code>ls verz</code>	← <i>POSIX</i>
<code>ls -l verz</code>	(mit zusätzlichen Informationen)
<code>dir verz</code>	← <i>MS CMD</i>

(wird hier `verz` ausgelassen: aktuelles Verzeichnis)

Erzeugen/Löschen von Verzeichnissen

Erzeugen eines Verzeichnisses *verz*:

`mkdir verz` ← *POSIX & MS CMD*

Löschen eines leeren(!) Verzeichnisses *verz*:

`rmdir verz` ← *POSIX & MS CMD*

Löschen eines Verzeichnisses *verz*
und **aller**(!) darin enthaltenen Verzeichnisse und Dateien:

`rm -rf verz` ← *POSIX*

`rmdir /S verz` ← *MS CMD*

Optionen beginnen mit **-** (POSIX) bzw. **/** (MS CMD),
variieren Verhalten eines Kommandos

Kommandos auf Dateien

Löschen von Datei *x*:

`rm x` ← *POSIX*
`del x` ← *MS CMD*

Verschieben von Datei/Verzeichnis *x* nach Datei/Verzeichnis *y*
(entspricht **Umbenennung**):

`mv x y` ← *POSIX*
`move x y` ← *MS CMD*

Kopie von Datei *x* nach Datei/Verzeichnis *y*:

`cp x y` ← *POSIX*
`copy x y` ← *MS CMD*

Viele weitere Kommandos sind in vielen Quellen beschrieben.

Empfehlung: Erfahrung mit Kommandozeile macht extrem effektiv!

Wildcards

Wildcards („Joker“) erlauben Zusammenfassung von Dateinamen:

? in Dateinamen: genau ein beliebiges Zeichen

***** in Dateinamen: beliebig viele beliebige Zeichen

Beispiele (formuliert für POSIX; MS CMD entsprechend):

```
rm F?t?.jpg      ← löscht Foto.jpg, Fete.jpg, Fati.jpg, ...  
rm *             ← löscht alle(!) Dateien in aktuellem Verz.  
mv *.java pm1-pt ← schiebt alle Java-Dateien in Verz. pm1-pt
```

Übung: Kommandozeile

Aufgabenblatt 01, Aufgabe 01

Editoren

zum Erstellen einfacher Textdateien dienen Editoren – „Schreibmaschinen“ ohne Formatierung oder grafische Elemente.

unterschiedliche Editoren auf jedem System, z.B.

- gedit, kwrite, leafpad, vi, emacs, ... auf POSIX
- TextEdit auf Apple macOS (Aufruf: `open -a TextEdit`)
- notepad, notepad++, ... auf MS Windows

Aufruf von der Kommandozeile, Ausführung i.d.R. in eigenem Fenster
erstellte Texte lassen sich (per Menü) in Dateien abspeichern

Hinweis: machen Sie sich mit „Ihrem“ Editor gut vertraut – das erspart Ihnen viel Zeit beim Programmieren

Programme

Programm „Hallo Welt!“

Inhalt der Textdatei Hallo.java ist ein Java-**Programm** Hallo:

```
1  /*
2   * Programm: gibt "Hallo Welt!" aus
3   */
4  public class Hallo {
5      public static void main(String[] args) {
6          // Ausgabe der Nachricht
7          System.out.println("Hallo Welt!");
8      }
9  }
```

in **Kommandozeile Übersetzung** und **Ausführung** des Programms:

```
$ javac Hallo.java
```

← Übersetzung (erzeugt Hallo.class)

```
$ java Hallo
```

← Ausführung (von Hallo.class)

```
Hallo Welt!
```

← Ausgabe des Programms

```
$
```

← neuer Prompt

Übung: Hello World

Aufgabenblatt 01, Aufgabe 02

Layout

White Space (Leerraumzeichen: Leerzeichen, Tab, Zeilenumbruch) können (fast) **beliebig** zwischen Codeelementen gesetzt werden.

Codelayout theoretisch frei, in Praxis **Konventionen** zugunsten Lesbarkeit und **Konsistenz**, z.B.:

- öffnende geschweifte Klammer { ans **Zeilenende** setzen
- schließende geschweifte Klammer } auf **eigene Zeile** setzen, unter **Anfang** der Zeile mit zugehöriger öffnender Klammer
- **zwischen** geschweiften Klammern **Einrückung** um **2 Leerzeichen**
- moderate **Zeilenlänge** einhalten (z.B. max. 100 Zeichen)
- **Leerzeilen** sparsam einsetzen

Hinweis: Codebeispiele folgen den Konventionen, dienen als Vorlage

Kommentare

Kommentare erlauben **Dokumentation** des Codes (für Entwickler)

Kommentare werden bei Übersetzung ignoriert (gelten als Leerzeichen)

es gibt zwei Formen von Kommentaren:

- **Blockkommentare**, mehrere Zeilen möglich: `/* ... */`

```
1 | /*  
2 |  * Programm: gibt "Hallo Welt!" aus  
3 |  */
```

- **Zeilenkommentare**, nur bis zum Zeilenende: `// ...`

```
6 | // Ausgabe der Nachricht
```

Hinweis: Kommentare **kurz, aussagekräftig, präzise** halten

Klassen

ein Java-Programm besteht aus min. einer **Klasse**:

```
public class Hallo {  
    public static void main(String[] args) {  
        System.out.println("Hallo Welt!");  
    }  
}
```

Klassen modellieren **Begriffe** ($\hat{=}$ **Substantive** in nat. Sprache),
z.B. Arten von Dingen, Zuständen, Zusammenhängen, ...

Klasse Hallo modelliert hier „Programm Hallo“

Namen von Klasse und Datei (hier: Hallo) **müssen übereinstimmen!**
Dateiendung ist .java

Methode main

min. eine Klasse eines Programms enthält eine **Methode main**:

```
public class Hallo {  
    public static void main(String[] args) {  
        System.out.println("Hallo Welt!");  
    }  
}
```

Methoden modellieren **Verhalten** ($\hat{=}$ **Verben** in nat. Sprache)

main (in Hallo) modelliert „Programm Hallo ausführen“

Kopf von main wird immer in der gezeigten Weise **deklariert**

Anweisungen

Methode enthalten beliebig viele (hier: eine) **Anweisungen**:

```
public class Hallo {  
    public static void main(String[] args) {  
        System.out.println("Hallo Welt!");  
    }  
}
```

Anweisungen modellieren **Aktionen** ($\hat{=}$ **Befehle** in nat. Sprache).

hier: “gib die Zeichenfolge

Hallo Welt!

(gefolgt von einem Zeilenumbruch) auf dem Bildschirm aus”

Syntaxfehler

Code kann unterschiedliche Arten **Fehler** enthalten.

Syntaxfehler sind **strukturelle Fehler** im Aufbau:

```
public class Hallo {  
    public static void main(String[] args) {  
        System.out.println("Hallo Welt!")  
    }  
}
```

Sie werden **bei Übersetzung** des Codes entdeckt:

```
$ javac Hallo.java  
Hallo.java:3: error: ';' expected  
    System.out.println("Hallo Welt!")  
                        ^  
  
1 error  
$
```

Fehler in natürlichen Sprachen

In **natürlichen** („menschlichen“, nicht-formalen) **Sprachen** kennen wir **Syntaxfehler** als **Rechtschreib-** oder **Grammatikfehler**:

Buchschtabe

Dietmar's Kiosk

Tu noch mal Curry bei!

Außerdem gibt es **Semantikfehler**:

Text ist zwar formal korrekt, ergibt aber **inhaltlich keinen Sinn**:

Dunkel war's, der Mond schien helle,
schneebedeckt die grüne Flur,
als ein Wagen blitzesschnelle
langsam um die Ecke fuhr.

Semantikfehler in **Programmen** entdeckt **Übersetzer** nur zum Teil (**Typfehler**), erfordern sonst **explizite Behandlung** (dazu später).

Klassen und Methoden

Deklaration einer Klasse

Deklaration zur Festlegung einer Klasse:

```
4 | public class Hallo {  
9 | }
```

Klassenkopf zur Beschreibung der Klasse:

public: **Zugriffsrecht** (dazu später) – vorerst stets **public**

class: festes **Schlüsselwort:** „Klassendeklaration“

Hallo: **Bezeichner** (Name) der Klasse

wichtig: Klasse *X* in Datei *X.java*

Klassenkörper als **Implementierung** der Klasse:

{...}: **Block** mit Deklarationen der Elemente der Klasse

Bezeichner

Bezeichner werden prinzipiell frei gewählt; sie beginnen mit einem **Buchstaben**, gefolgt von beliebig vielen **Buchstaben oder Ziffern**:

Hallo main Hallo1 Hallo24h H hallo

Konvention: **Klassenbezeichner** beginnt mit **Großbuchstaben**

Konvention: für **zusammengesetzte Bezeichner**: **CamelCase**
(jeder angesetzte Wortteil beginnt mit Großbuchstaben)

HalloUndTschuess GradCelsius SehrLangeNamenSindNichtSoGut

Klasse mit mehreren Methoden

Klassen können beliebig viele Methoden enthalten:

```
1 public class HalloMitMethode {  
2     public static void gruesse() {  
3         System.out.println("Hallo Welt!");  
4     }  
5     public static void main(String[] args) {  
6         gruesse();  
7     }  
8 }
```

weitere Methoden werden auf **selber Ebene wie main** deklariert

ausgehend von **main** können Methoden **ausgeführt** werden:

Aufruf `gruesse()` bedeutet: „führe Methode an dieser Stelle aus“

Deklaration einer Methode

```
2 | public static void gruesse() {  
4 | }
```

Methodenkopf zur **Beschreibung** der Methode:

- public**: **Zugriffsrecht** (dazu später) – vorerst stets **public**
- static**: **Art** (später) – vorerst stets **static** (**Klassenmethode**)
- void**: **Ergebnistyp** (gleich) – **void** bedeutet „kein Ergebnis“
- gruesse**: **Bezeichner** der Methode – von Entwickler gewählt
- Konvention**: zuerst **Kleinbuchstaben**, dann **CamelCase**
- (...)**: **Parameterliste** (gleich) – **()** bedeutet „keine Param.“

Methodenkörper als **Implementierung** der Methode:

{...}: **Block** mit Anweisungen, bilden Verhalten der Methode

Werte und Typen

einzelne verarbeitete **Daten** heißen **Werte**

in Java **Werte unterschiedlicher Art**, z.B.:

Zeichenketten, einzelne Zeichen, Zahlen, Dreiecke, Personen, ...

ein **Typ** beschreibt eine **Art von Werten**; **jeder Wert hat einen Typ**

in Java: wo ein Wert erwartet wird, muss sein Typ **deklariert** sein
(siehe folgendes Beispiel)

Methode mit Ergebnis

```
1 public class HalloMitMethodeMitRet {  
2     public static String gruss() {  
3         return "Hallo Welt!";  
4     }  
5     public static void main(String[] args) {  
6         System.out.println(gruss());  
7     }  
8 }
```

Methode gruss liefert hier ein **Ergebnis** (auch: **Rückgabewert**):

String (Z.2) bestimmt **Ergebnistyp** (hier: **Zeichenkette**)

return (Z.3) bestimmt **Ergebniswert** vom **Ergebnistyp** und
beendet (als Anweisung) die **Ausführung** der Methode

Ergebnis ist **Wert** des Aufrufs **gruss()**

„Produzieren“ von Werten

Terminologie: eine Methode kann, unabhängig voneinander,

- einen **Ergebniswert** mit `return` **zurückgeben**
⇒ Wert in aufrufender Methode verwendbar
- beliebige Werte z.B. auf dem Bildschirm **ausgeben**
⇒ Ausgabe nur für Nutzer, nicht im Programm verwendbar

Ausgaben erfolgen nur, wenn/wo **explizit gefordert**

→ i.d.R. separate Methoden für Berechnungen und für Ausgaben

Separierung durch Methoden

Methode `gruss` allein bestimmt den Gruß, alternativ in Z.3 z.B.:

```
return "Hallo Welt!";           // 1. Variante  
return "Hi Welt, altes Haus!"; // 2. Variante  
return "Ciao Welt!";           // 3. Variante
```

Aufruf bleibt jeweils gleich:

```
System.out.println(gruss());
```

dagegen bestimmt allein Methode `main` **Verwendung** des Grußes
(hier: Ausgabe auf Bildschirm)

Separierung ist wesentliches **Entwurfsziel** (Bausteinprinzip)

Methode mit Parameter

```
1 public class HalloMitMethodeMitPar {  
2     public static String gruss(String name) {  
3         return "Hallo " + name + "!";  
4     }  
5     public static void main(String[] args) {  
6         System.out.println(gruss("Welt"));  
7     }  
8 }
```

Methode gruss hat hier einen **Parameter name** vom Typ String
bei Aufruf von gruss wird das **Argument "Welt"** **übergeben**,
d.h. Parameter **name** erhält **Wert "Welt"** des Arguments
bei Ausführung von gruss **repräsentiert name** diesen Wert
(**Operator +** fügt Zeichenketten zusammen)

Verwendung von Parametern

Parametername ist **nur in** der entsprechenden **Methode** bekannt
(z.B. Parameter name nur in gruss, nicht in main)

Konvention: zuerst **Kleinbuchstabe**, dann **CamelCase**
(wie Methodename, aber ohne folgende öffnende Klammer)

mehrere Parameter in Deklaration möglich, getrennt durch **Komma**:

```
public static String gruss(String vorname, String nachname) {  
    ...  
}
```

bei Aufruf müssen **Anzahl**, **Typen** und **Reihenfolge** der Argumente mit denen der Parameter **übereinstimmen**:

```
System.out.println(gruss("James", "Bond"));
```

Wiederverwendung durch Methoden

Methode `gruss` allein bestimmt die Grußformel

verschiedene Aufrufe ergeben verschiedene Grüße, in Z.6 z.B.:

```
System.out.println(gruss("Welt"));    // "Hallo Welt!"  
System.out.println(gruss("Tina"));    // "Hallo Tina!"  
System.out.println(gruss("Sie"));     // "Hallo Sie!"
```

Methode `main` allein bestimmt

- die Verwendung des Grußes (hier: Ausgabe auf Bildschirm)
- die/den Angesprochene/n (Welt, Tina, Sie, ...)

Wiederverwendung ist wesentliches Entwurfsziel (Bausteinprinzip)

Programm mit mehreren Klassen

ein Programm kann mehr als eine Klasse / Datei enthalten:

```
1 public class Kontakt {  
2     public static String gruss(String name) {  
3         return "Hallo " + name + "!";  
4     }  
5 }
```

```
1 public class HalloMitKontakt {  
2     public static void main(String[] args) {  
3         System.out.println(Kontakt.gruss("Welt"));  
4     }  
5 }
```

\$ javac HalloMitKontakt.java Kontakt.java ← *alle übersetzen*

\$ java HalloMitKontakt ← *main dieser Klasse ausführen*

Hallo Welt!

\$

Aufruf einer Methode einer anderen Klasse

Aufruf von Methode **anderer Klasse** durch **Qualifizierung** mit Klassennamen:

```
3 | System.out.println(Kontakt.gruss("Welt"));
```

HalloMitKontakt.java

Aufruf von Methode **selber Klasse** meist ohne Qualifizierung:

```
6 | System.out.println(gruss("Welt"));
```

HalloMitMethodeMitPar.java

Übung: Communication

Aufgabenblatt 01, Aufgabe 03

Klassen- vs. Instanzmethoden

Es gibt in Java zwei Arten von Methoden:

- **Klassenmethoden** werden über Klassen aufgerufen, beschreiben **generelles** Verhalten
z.B. `Kontakt.gruss("Welt")`
- **Instanzmethoden** werden über Werte aufgerufen, beschreiben **anwendungsabhängiges** Verhalten
z.B. `System.out.println("Hallo")`

(später mehr dazu)

Wo der Unterschied keine Rolle spielt, spricht man nur von **Methoden**.

Variablen

Lokale Variablen

Variable: Speicherplatz bestimmten Typs mit Bezeichner

lokale Variablen werden in Methoden deklariert:

```
1 public class KontaktMitVar {  
2     public static String gruss(String name) {  
3         String s = "Hallo ";  
4         s = s + name;  
5         s = s + "!";  
6         return s;  
7     }  
8 }
```

lokale Variablen werden

- bei jedem Methodenaufruf neu angelegt
- als temporäre „Hilfsspeicher“ verwendet
- bei Ende der Ausführung der Methode eliminiert

Deklaration lokaler Variablen

```
3 | String s = "Hallo ";
```

KontaktMitVar.java

String: Typ der Variable

s: **Bezeichner** – von Entwickler gewählt

Konvention: Kleinbuchstabe und CamelCase
(wie Parametername)

= ...: **Initialisierung** mit **Startwert** (optional)

kein Zugriffsrecht: lok. Variable nur in Methode existent/verwendbar

Zuweisung $v = w$ ersetzt Wert der Variablen v durch Wert von w

Verwendung lokaler Variablen

vorheriges Beispiel wenig sinnvoll: Code wird länger, kein Vorteil

Faustregel: lokale Variablen nur verwenden, wenn mehrfach benutzt

```
1 public class KontaktAlsSheldon {  
2     public static String gruss(String name) {  
3         String s = "<knock> ";  
4         s = s + s + s + name + "! ";  
5         s = s + s + s;  
6         return s;  
7     }  
8 }
```

Frage: was ergibt `KontaktAlsSheldon.gruss("Penny")`?

Konventionen für Bezeichner:

- öffentlich (public) → aussagekräftig: `KontaktAlsSheldon`
- intern (Parameter, lokale Variable) → kurz: `s`

Anweisungen und Kontrollstrukturen

Anweisungen

alle 3 Arten von **Anweisung** („Befehl“/**Statement**) bekannt:

- **Ausdrucksanweisung** (**Ausdruck** + Semikolon), z.B.:

- Aufruf einer Methode als Anweisung:

```
System.out.println("Hallo Welt!");
```

- Zuweisung als Anweisung:

```
s = s + name;
```

- **Kontrollanweisung**, z.B. return-Anweisung:

```
return "Hallo Welt!";
```

- **Deklarationsanweisung**:

```
String s = "Hallo ";
```

Konvention: Zeilenumbruch nach jeder Anweisung

Fallunterscheidung mit if-else

weitere Kontrollanweisung: **Fallunterscheidung if-else**

```
1 public class KontaktMitIfElse {  
2     public static String gruss(String sprache, String name) {  
3         String formel;  
4         if (sprache.equals("Deutsch")) {  
5             formel = "Hallo";  
6         } else {  
7             formel = "Hello";  
8         }  
9         return formel + " " + name + "!";  
10    }  
11 }
```

Aufruf z.B.: `KontaktMitIfElse.gruss("Englisch", "Martha")`

Fallunterscheidung mit if-else: Ausführung

sprache.equals("Deutsch") vergleicht Zeichenketten
sprache und "Deutsch": wahr wenn gleich, sonst falsch

```
4   if (sprache.equals("Deutsch")) {  
5       formel = "Hallo";  
6   } else {  
7       formel = "Hello";  
8   }
```

KontaktMitIfElse.java

ist **Bedingung** (nach if) wahr, wird **erster Block {...}** ausgeführt
ist sie falsch, wird **zweiter Block** (nach else) ausgeführt

→ damit Möglichkeit, Entscheidungen zu fällen

Fallunterscheidung mit if-else: Schreibweise

Konventionen:

- Leerzeichen zwischen if und öffnender Klammer (ist keine Methode!)

```
4|     if (sprache.equals("Deutsch")) {
```

- else hinter schließender Blockklammer zu if

```
6|     } else {
```

Fallunterscheidung mit if

einfachere Variante (hier gleicher Effekt): **if-Anweisung** (ohne else)

```
1 public class KontaktMitIf {  
2     public static String gruss(String sprache, String name) {  
3         String formel = "Hello";  
4         if (sprache.equals("Deutsch")) {  
5             formel = "Hallo";  
6         }  
7         return formel + " " + name + "!";  
8     }  
9 }
```

ist **Bedingung falsch**, wird Block nach if **übersprungen** (zu Z.7) –
entspricht der Verwendung von else mit leerem Block (statt Z.6):

```
...  
} else {  
}  
...
```

Fallunterscheidung mit if-else: Beispiel max

```
1 public class Mathe1 {  
2     public static int max(int a, int b) {  
3         if (a >= b) {  
4             return a;  
5         } else {  
6             return b;  
7         }  
8     }  
    // ...  
68 }
```

Typ **int** beschreibt **ganzzahlige Werte**; Operator **>=** entspricht $\geq$

Mathe1.max(5, 3) oder Mathe1.max(3, 5) ergibt Wert 5

Hinweis: Methode in Java bereitgestellt als Math.max(...),
entsprechend auch Math.min(...)

Schachtelung von Fallunterscheidungen

Anweisungen in Fallunterscheidungen können wieder Fallunterscheidungen sein (**Schachtelung**):

```
9 | public static String signum(int n) {  
11 |     if (n < 0) {  
12 |         return "negativ";  
13 |     } else {  
14 |         if (n > 0) {  
15 |             return "positiv";  
16 |         } else {  
17 |             return "neutral";  
18 |         }  
19 |     }  
29 | }
```

return im if-Block: else optional

else kann entfallen, wenn if-Block mit return schließt
(da dann Methode beendet, Fortsetzung ohnehin nur für else-Fall)

```
9 | public static String signum(int n) {  
22 |     if (n < 0) {  
23 |         return "negativ";  
24 |     }  
25 |     if (n > 0) {  
26 |         return "positiv";  
27 |     }  
28 |     return "neutral";  
29 | }
```


Methode testen

Wenn **unterschiedliche Aufrufe/Ergebnisse** von Methode möglich, **mehrfache Aufrufe** nötig, um **Korrektheit** zu prüfen.

einfache Vorgehensweise (effektivere später):

```
1 public class Mathe1Test {
2     public static void main(String[] args) {
3         System.out.println(Mathe1.max( 0,-1)); // 0
4         System.out.println(Mathe1.max(-1,-2)); // -1
5         System.out.println(Mathe1.max( 1, 1)); // 1
6         // ...
7         System.out.println(Mathe1.signum(-8)); // 8
8         System.out.println(Mathe1.signum( 3)); // 3
9         System.out.println(Mathe1.signum( 0)); // 0
10        // ...
11    }
12 }
```

Achtung: nur Stichproben, daher **keine Garantie der Korrektheit!**

Übung: abs + max3

Aufgabenblatt 02, Aufgabe 01

Rekursion: Beispiel Summe

mit Fallunterscheidung **rekursive** (rückläufige) Berechnung möglich

Beispiel: math. Formulierung der **Summe**

$$\sum_{i=1}^n i = 1 + 2 + \dots + n = \begin{cases} 0 & \text{für } n = 0 \\ \left(\sum_{i=1}^{n-1} i \right) + n & \text{sonst} \end{cases}$$

in Java:

```
30 public static int summeRek(int n) {  
31     if (n == 0) {  
32         return 0;  
33     } else {  
34         return summeRek(n - 1) + n;  
35     }  
36 }
```

Mathe1.java

Iteration mit while: Beispiel Summe

Iteration: wiederholte Ausführung derselben Anweisung(en)

while-Schleife: ... solange Bedingung erfüllt

Summe alternativ als schrittweise Umsetzung der Formulierung

$$\sum_{i=1}^n i = 1 + 2 + \dots + n$$

```
37 public static int summeLoop(int n) {  
38     int s = 0;  
39     int i = 1;  
40     while (i <= n) {  
41         s = s + i;  
42         i = i + 1;  
43     }  
44     return s;  
45 }
```

Iteration mit while: Aufbau

Struktur von while entspricht der von if (ohne else-Teil):

```
40 while (i <= n) {  
41     s = s + i;  
42     i = i + 1;  
43 }
```

anders als für if: nach Ausführung des Blocks wird **erneut** die Bedingung **geprüft**, ggf. der Block wieder ausgeführt

wie für if: wenn die **Bedingung nicht zutrifft**, wird die **Schleife beendet** und mit der folgenden Anweisung fortgesetzt

Iteration mit while: Beispiel Quadrate

Methoden können auch der Ausgabe dienen:

```
46 public static void quadrateAusgabe(int n) {  
47     int i = 1;  
48     while (i <= n) {  
49         System.out.println(i * i);  
50         i = i + 1;  
51     }  
52 }
```

Mathe1.java

Aufruf `Mathe1.quadrateAusgabe(4)` produziert die Ausgabe:

1
4
9
16

Übung: fak + quadrateAusgabeRueck

Aufgabenblatt 02, Aufgabe 02

Iteration mit while: Beispiel Zeichenkettenbau

```
2  public static String wiederhole(String s, int n) {  
3      String r = "";  
4      int i = 0;  
5      while (i < n) {  
6          r = r + s;  
7          i = i + 1;  
8      }  
9      return r;  
10 }
```

Zeichenkette1.java

Beispiel:

"Tra" + Zeichenkette1.wiederhole("la", 4) + "!"
produziert: "Tralalalala!"

Übung: grussMitZeit + zitronenEis

Aufgabenblatt 02, Aufgabe 03

Schachtelung von Iterationen

Schleife ist Anweisung, also Schleife in Schleife möglich:

```
53 public static void ausgabeAlsQuadrat(int n) {  
54     int x = 1;  
55     int i = 1;  
56     while (i <= n) {  
57         int j = 1;  
58         while (j <= n) {  
59             System.out.print(x);    // ohne Zeilenumbruch  
60             System.out.print(" "); // ohne Zeilenumbruch  
61             x = x + 1;  
62             j = j + 1;  
63         }  
64         System.out.println();      // reiner Zeilenumbruch  
65         i = i + 1;  
66     }  
67 }
```

Hinweis: elegantere Formulierungen werden später vorgestellt

Typen und Ausdrücke

Ausdrücke

ein **Ausdruck** repräsentiert einen **Wert** als:

- **Literal** (**Konstante**):

"Hallo!" 17

- **Variablen-/Parametername**:

name formel s

- **Methodenaufruf**:

gruss("Welt") Mathe1.max(3,7)

- **Operation**

formel + " " + name a >= b i = i + 1

jeder **Ausdruck** hat einen **eindeutigen Typ und Wert**

Auswertung eines Ausdrucks: Bestimmung des dargestellten Wertes

Operationen auf beliebigen Typen: Priorisierung

unäre Operatoren verknüpfen einen Wert, **binäre Operatoren** zwei Werte (**Operanden**) bestimmten Typs zu einem Ausdruck

einige Operatoren sind für Werte aller Typen definiert:

Klammerung: (a)

- hat selben Typ und Wert wie a
- Festlegung oder Verdeutlichung der Reihenfolge der Auswertung:
 $(a + b) * c$

Op. auf beliebigen Typen: Zuweisung / Vergleich

Konvention: Leerzeichen vor und hinter binären Operatoren

Zuweisung: $a = b$

- Ausdruck a muss ein Variablenname sein
- a und b müssen selben (oder kompatiblen) Typ haben
- Wert b wird in die Variable kopiert („von rechts nach links“)
- vorheriger Wert der Variable ist dann überschrieben / verloren(!)
- Wert des Ausdrucks ist zugewiesener Wert – somit möglich:

$$a = b = c$$

weist Wert von c erst b , dann als Ergebnis a zu:

$$a = (b = c)$$

Vergleich auf Gleichheit / Ungleichheit: $a == b$ / $a != b$

- Resultat ist „wahr“ / „falsch“

Nebeneffekt

Nebeneffekt (**side effect**) eines Ausdrucks: **Zustandsänderung** bei Auswertung, neben der eigentlichen Produktion eines Wertes

Beispiele:

- Änderung des Wertes einer Variablen (z.B. $a = b$)
- Ausgabe (in Methodenaufruf)
- Fehler (zu dem dadurch verursachten Effekt später)
- Ausführungskosten

Nur **wenige Operatoren** haben Nebeneffekte,
aber **potentiell jeder(!) Methodenaufruf**.

void

void ist Platzhalter für **Rückgabetyp** von Methode, die **keinen Wert** produziert, also **nur Nebeneffekt** hat (z.B. quadrateAusgabe)

void-Ausdruck kann nicht als Wert verwendet werden:

```
System.out.println(Mathe1.quadrateAusgabe(4)); // Fehler!
```

sondern nur in Ausdrucksanweisung:

```
public static void main(String[] args) {  
    // ...  
    Mathe1.quadrateAusgabe(4);  
    // ...  
}
```

Anmerkung: In anderen Sprachen nennt man eine

- Methode ohne Wert (nur Nebeneffekte): **Prozedur**
- Methode ohne Nebeneffekt (nur Wert): (**pure**) **Funktion**

return; in void-Methode

(nur) void-Methoden benötigen keine return-Anweisung – sie enden automatisch am Ende ihres Körpers (Blocks)

Anweisung **return;** (ohne Wert) beendet void-Methode vorzeitig:

```
if (alter < 18) {  
    return;  
}  
System.out.println("Wählen Sie eine Partei:");
```

Verwendung **nur in Fallunterscheidungen** sinnvoll

Zeichenketten

Zeichenketten haben Typ **String**

Literale dieses Typs stehen zwischen **Anführungszeichen**, z.B.:

```
"!tleW ollaH"
```

```
"Was, 3*4=10? So'n Quatsch!"
```

```
"X"
```

```
" "           // leere Zeichenkette
```

Zeichenkette können beliebig viele – auch 0 – Zeichen enthalten.

Anführungszeichen in einer Zeichenkette werden als `\"` dargestellt:

```
"\"Verboten verboten!\" stand dort"
```

Zeichenketten: Operationen

Operationen auf Zeichenketten:

- $s1 + s2$ hängt $s1$ und $s2$ zusammen, ergibt String-Wert

```
"Frei" + "tag"    // Wert ist "Freitag"
```

- $s1.equals(s2)$ vergleicht $s1$ und $s2$, ergibt Wahrheitswert

```
("Frei" + "tag").equals("Freitag") // Wert ist „wahr“
```

```
"rennen".equals("Rennen")          // Wert ist „falsch“
```

wichtig: für Zeichenketten haben

$s1 == s2$

$s1 != s2$

eine andere Wirkung (Erklärung später) – **nicht verwenden!**

zusammengesetzter Zuweisungsausdruck

häufige Operation: $s1 = s1 + s2$

dafür Kurzschreibweise: $s1 += s2$

(auch hier ist der Wert des Ausdrucks der $s1$ neu zugewiesene Wert)

Beispiel: der Ausdruck

```
s = s + "!"
```

entspricht

```
s += "!"
```

Einfache vs. zusammengesetzte Typen

String ist **zusammengesetzter/komplexer Typ**: mit Mitteln der Sprache **aus einfacheren Werten** (hier: einzelnen Zeichen) aufgebaut (Instanzen solcher Typen heißen **Objekte** – dazu später viel mehr)

einfacher/primitiver Typ: nicht zusammengesetzt, sondern **direkt von Sprache** bereitgestellt; beschreibt einzelne, elementare Werte

Weitere **Operationen** sind **nur für einfache Typen** definiert, nicht auf zusammengesetzte Typen anwendbar.

Zeichen

einzelne **Zeichen** haben Typ **char**

Literale dieses Typs stehen zwischen **Apostrophen**, z.B.:

'a' 'A' '0' '*' '.' ' ' // Leerzeichen

einzelne Zeichen werden i.d.R. nur bei Verarbeitung von Zeichenketten durch entsprechende Methoden verwendet (später)

Verwendung meist in Zeichenketten: **operator +** mit

- einem Argument vom Typ String
- einem Argument vom Typ char

hängt Argumente von links nach rechts zu Zeichenkette zusammen:

" " + 'H' + 'a' + 'l' + 'l' + 'o' // Wert ist "Hallo"

Zeichen: Zeilenumbruch

nicht direkt darstellbares Zeichen: **Zeilenumbruch** `'\n'`

- **einzelnes Zeichen**, obwohl im Code durch zwei dargestellt (in `\n` steht `\` für „Sonderzeichen“, `n` für „newline“)
- bedeutet **Ende der Zeile und Beginn neuer Zeile** („Enter-Taste“)
- auch in Zeichenketten verwendbar – analog zu `'\"'`

Beispiel:

```
System.out.println("Hallo\nWelt!");
```

produziert Ausgabe:

```
Hallo  
Welt!
```

(kein Leerzeichen nach Hallo)

Übung: Zeichenketten.wiederhole

Aufgabenblatt 02, Aufgabe 04

Ganzzahlwerte

Ganzzahlwerte haben i.d.R. Typ **int** (weitere Varianten später)

Literale dieses Typs sind (Dezimal)Ziffernfolgen:

26 456413272 1 0

(weitere Darstellungsvarianten später)

negative Zahlen als Ergebnis des unären **Operators** -:

-2 -(-5)

Vergleiche zweier Werte ergeben „wahr“ oder „falsch“:

```
i1 < i2 // kleiner als
```

```
i1 > i2 // groesser als
```

```
i1 <= i2 // kleiner als oder gleich
```

```
i1 >= i2 // groesser als oder gleich
```

Ganzzahlwerte: arithmetische Operationen

arithmetische Operationen mit `int`-Werten ergeben `int`-Werte:

```
i1 + i2    // Summe
i1 - i2    // Differenz
i1 * i2    // Produkt
i1 / i2    // ganzwertiger Quotient
i1 % i2    // ganzwertiger Modulo (Erklärung folgt)
-i         // additives Inverses / Negation / Vorzeichenwechsel
```

Beispiel:

```
1 public class Mathe {
2     public static int mittel(int a, int b) {
3         return (a + b) / 2;
4     }
    // ...
29 }
```

Ganzzahlwerte: Division und Rest

Division ganzer Werte ergibt **ganzzahligen „Teiler“**: $13 / 5 == 2$

Modulo-Operator % ergibt **ganzzahligen Rest**: $13 \% 5 == 3$

Hinweis: auch für negative Werte def., **nicht wie in Mathematik**:

$(a / b) * b + (a \% b) == a$ **// bitte nicht merken! :-)**

Anwendung: Bestimmung von **Teilbarkeit**

$a \% b == 0$ (bzw. $!= 0$) heißt „a ist (nicht) teilbar durch b“

```
if (n % 2 == 0) {  
    // n ist gerade  
} else {  
    // n ist ungerade  
}
```

Ganzzahlwerte: Zuweisungsausdrücke

arithmetische Ausdrücke der Form

$$x = x + y$$

$$x = x - y$$

$$x = x * y$$

$$x = x / y$$

$$x = x \% y$$

sind darstellbar als **zusammengesetzte Zuweisungsausdrücke**

$$x \text{ += } y$$

$$x \text{ -= } y$$

$$x \text{ *= } y$$

$$x \text{ /= } y$$

$$x \text{ \%=} y$$

Ganzzahlwerte: Inkrement / Dekrement

häufige Operation (z.B. in Schleifen): $x += 1$ oder $x -= 1$

dafür Kurzschreibweise mit (**unären**) **Präfix**-Operatoren:

$++x$ erhöht Wert von x um 1 (**Inkrement**)

$--x$ verringert Wert von x um 1 (**Dekrement**)

Wert der Ausdrücke ist der von x **nach Erhöhung/Verringerung**

```
14 public static int summeLoop(int n) {  
15     int s = 0;  
16     int i = 1;  
17     while (i <= n) {  
18         s += i;  
19         ++i;  
20     }  
21     return s;  
22 }
```

Gleitkommawerte

Gleitkommawerte („Kommawerte“) haben i.d.R. Typ **double**

Literale sind Ziffernfolgen mit **Dezimalpunkt** (**Varianten** später):

4.27 2294.1 0.0

Literale sind **stets nichtnegativ**, negative Werte durch **Operator -**

Vergleichsoperationen **< > <= >=** und **arithmetische Op. + - ***
sind definiert wie für **int**

/ liefert für **double**-Werte tatsächlichen Quotienten:

13.0 / 5.0 hat den Wert 2.6

% ist ebenfalls definiert, wird aber **sehr selten** verwendet

analog: **Zuweisungsoperatoren += -= *= /= %=**

Gleitkommawerte: Rundungsfehler

in Operationen auf Gleitkommawerten eventuell

Rundungsabweichungen:

mathematisch gleiche Werte sind ggf. nicht identisch dargestellt

Beispiel: $0.1 + 0.1 + 0.1$ und 0.3 sind verschieden!

Probe:

```
System.out.println(0.1 + 0.1 + 0.1); // 0.30000000000000004  
System.out.println(0.3);             // 0.3
```

⇒ für Gleitkommawerte $d1$ und $d2$ sind die Ausdrücke

$d1 == d2$ und $d1 != d2$

legal, aber meist nicht sinnvoll

Zahltypen: Math-Methoden

Klasse **Math** bietet viele Klassenmethoden
für häufige mathematische Operationen

für **Ganz-** und für **Gleitkommazahlwerte** *a* und *b*:

```
Math.min(a,b)    // kleinerer der Werte a und b  
Math.max(a,b)    // groesserer der Werte a und b  
Math.abs(a)      // Betrag von a
```

viele weitere Methoden für Gleitkommawerte: `sqrt`, `log`, `sin`, ...

Hinweis: keine Methode für Vergleich bei Rundungsabweichungen

Wahrheitswerte

Wahrheitswerte / logische Werte haben Typ **boolean**

Literale dieses Typs und einzig mögliche Werte: **true** und **false**

auch dieser Typ wird verwendet zur Deklaration von Variablen:

```
boolean b = "Hallo".equals("Hello"); // false
boolean istKleiner = x < y; // Vergleich ergibt Wahrheitswert
```

und als Parameter- oder Rückgabety von Methoden:

```
public static boolean istEnglisch(String gruss) {
    return gruss.equals("Hello");
}

public static String neuesPasswort(boolean mitHinweis) {
    if (mitHinweis) {
        System.out.println("Passwort benoetigt min. 8 Zeichen");
    }
    // ...
}
```

Wahrheitswerte: Beispiel Wertebereich

beliebig komplexe Wahrheitsaussagen als Methoden:

```
public static boolean istZwischen(int x, int y, int z) {  
    if (x <= y) {  
        if (y <= z) {  
            return true; // y liegt im Bereich [x;z]  
        }  
    }  
    return false; // alle anderen Fälle  
}
```

Aufruf:

```
istZwischen(1, 7, 10) // Wert: true  
istZwischen(0, -3, 7) // Wert: false
```

wichtig: $x \leq y \leq z$ nicht möglich!

wie $(x \leq y) \leq z$: Vergleich von Wahrheitswert und Ganzzahl

Logische Ausdrücke

logische Ausdrücke werden v.a. in Bedingungen verwendet:

```
if (istZwischen(x, y, z)) {  
    max = z;  
}
```

außer Literalen, Variablen, Methodenaufrufen auch Operationen:

- Operationen auf anderen Typen, die boolean-Werte produzieren (v.a. Gleichheits- und Vergleichsoperatoren)
- Operationen mit logischen Operatoren auf logischen Werten

Logische Operatoren

logischer Operator hat boolean-Werte als **Operand** u. **Ergebnis**:

- **$!b$** (log. Negation) ist true genau dann, wenn **b** nicht true:

```
istTag = !istNacht;
```

- **$b1 \ \& \ b2$** (log. Und) ist true g.d.w. **$b1$** und **$b2$** true:

```
istLVtag = istVorlesungszeit & istWerktag;
```

- **$b1 \ | \ b2$** (log. Inklusiv-Oder) ist true g.d.w.
 $b1$ oder **$b2$** (oder beide) true:

```
istFrei = istWochenende | istFeiertag;
```

- **$b1 \ ^ \ b2$** (log. Exklusiv-Oder) ist true g.d.w.
entweder $b1$ oder $b2$ (nicht beide!) true:

```
istEinstimmig = (anzahlJa > 0) ^ (anzahlNein > 0);
```

Hinweis: $a \ ^ \ b$ für Wahrheitswerte wie $a \ != \ b$

Logische Operatoren: Wertetabellen

da nur zwei Wahrheitswerte, log. Operatoren explizit beschreibbar:

log. Negation

b	$!b$
false	true
true	false

log. Und

$a \ \& \ b$	false	true
false	false	false
true	false	true

log. Inklusiv-Oder

$a \ \ b$	false	true
false	false	true
true	true	true

log. Exklusiv-Oder

$a \ \wedge \ b$	false	true
false	false	true
true	true	false

entsprechende Zuweisungsoperatoren: $\&=$ $|=$ $\wedge=$

Logische Ausdrücke: Beispiel Wertebereich

```
if (x <= y) {  
    if (y <= z) {  
        return true;  
    }  
}  
return false;
```

kann ersetzt werden durch:

```
if (x <= y & y <= z) {  
    return true;  
}  
return false;
```

bzw. einfach durch:

```
return x <= y & y <= z;
```

Übung: Wertebereiche

Aufgabenblatt 02, Aufgabe 05

für Wahrheitswerte **a** und **b** sind äquivalent:

```
if (a) {  
} else {  
    // X  
}
```

```
if (!a) {  
    // X  
}
```

```
if (a) {  
    if (b) {  
        // Y  
    }  
}
```

```
if (a & b) {  
    // Y  
}
```

```
if (a) {  
    // Z  
} else {  
    if (b) {  
        // (auch) Z  
    }  
}
```

```
if (a | b) {  
    // Z  
}
```


Logische Ausdrücke: Anwendung

beliebig komplexe logische Ausdrücke sind möglich:

```
boolean istArbeitszeit =    (  wochentag <= 5
                             & istZwischen(8, stunde, 17)
                             & !istUrlaub)
    | istSelbstaendig;
```

Konditionaler Operator

konditionaler Operator ?: („Elvis“) mit 3 Operanden:

LogikAusdruck ? *Ausdruck1* : *Ausdruck2*

vollzieht **bedingte Auswertung**: falls Wert von *LogikAusdruck*

true: *Ausdruck1* auswerten, Wert ist Resultat

false: *Ausdruck2* auswerten, Wert ist Resultat

wichtig:

- nur der gewählte Ausdruck wird ausgewertet
- ähnlich wie `if-else`, jedoch **Ausdruck** statt **Anweisung**!

Beispiel:

```
(sprache.equals("Deutsch") ? "Hallo " : "Hello ") + name + "!"
```

Konditionaler Operator: Beispiel

Fallunterscheidung zwischen **zwei Anweisungen gleicher Art** mit **Unterschied nur im verarbeiteten Wert**:

```
2 public static int max(int a, int b) {  
3     if (a >= b) {  
4         return a;  
5     } else {  
6         return b;  
7     }  
8 }
```

Mathe1.java

alternativ **eine Anweisung** mit **bedingtem Ausdruck**:

```
8 public static int max(int a, int b) {  
9     return a >= b ? a : b;  
10 }
```

Mathe.java

Bedingte logische Operatoren && und ||

Varianten `&&` und `||` zu `&` und `|` mit **bedingter Auswertung**:

- ❶ werte den ersten (linken) Operanden aus
- ❷ wenn dessen Wert bereits das Ergebnis bestimmt, also:
 - für `&&`: `false`
 - für `||`: `true`

dann **beende Auswertung** mit diesem Wert

- ❸ **nur wenn nötig(!)**: werte zweiten (rechten) Operanden aus
- ❹ dessen Wert ist dann das Resultat

durch `&` und `|` dagegen **unbedingte Auswertung** beider Argumente

Hinweis: `(a && b)` entspricht `(a ? b : false)`

`(a || b)` entspricht `(a ? true : b)`

Logische Operatoren && und || vs. & und |

Unterschied bei **Nebeneffekt** im 2. Operand, z.B. Variablenänderung:

```
0 < a & a < ++b // Wert von b stets erhöht
```

```
0 < a && a < ++b // Wert von b ggf. unverändert
```

... oder **möglichem Fehler** (oder **Methodenaufruf** oder ...):

```
if (a != 0 & (b / a) > 3) { ... } // ggf. Div. durch 0!
```

```
if (a != 0 && (b / a) > 3) { ... } // korrekt
```

Faustregel:

- && / || (nur) verwenden, wenn
 - **Nebeneffekt** im 2. Operand, der
 - abhängig vom 1. Operand ggf. **verhindert werden** soll
- & / | **sonst** verwenden:
 - **invarianter Ablauf** (besser optimierbar)
 - **parallele** statt **sequentieller Auswertung** möglich

⇒ ggf. effizienter

Logische Operatoren && und ||: Entsprechungen

wenn **b** Ausdruck mit **Nebeneffekt** ist, gelten die folgenden Entsprechungen zu if-else nur mit && und || statt & und |:

```
if (a) {  
    if (b) {  
        // Y  
    }  
}
```

```
if (a && b) {  
    // Y  
}
```

```
if (a) {  
    // Z  
} else {  
    if (b) {  
        // (auch) Z  
    }  
}
```

```
if (a || b) {  
    // Z  
}
```

Übung: Datum.istSchaltjahr

Aufgabenblatt 03, Aufgabe 01

Anwendung: Vergleich von Gleitkommawerten

Aufgabe: Vergleich von zwei Gleitkommawerten,
dabei kleine Abweichungen durch Rundungsfehler ignorieren

Ansatz: Vergleich mit Toleranz:
 x „ungefähr gleich“ y , wenn Unterschied von x und y „klein“

Problem: wieviel ist „klein“?

Beispiel: 1g sind

- wenig bei Körpergewicht einer Person
- viel bei Medikamentdosierung

Vergleich mit Rundungstoleranz

Lösung: setze Abweichung in Bezug zur Größe der Werte, betrachte **relative Abweichung** statt **absoluter**:

$$\text{relative Abweichung} = \frac{\text{absolute Abweichung}}{\text{Betrag der Werte}}$$

$x \sim y$ („ungefähr gleich“), wenn **relative Abweichung** kleiner als (hinreichend klein gewählter) Wert ϵ , z.B. $\epsilon = 0.000001$:

$$\frac{|x - y|}{\max\{|x|, |y|\}} < \epsilon \quad \text{falls } x, y \neq 0$$

Hinweis: gewünschte **Genauigkeit** ϵ ist **anwendungsabhängig**!

Vergleich mit Rundungstoleranz: Implementierung

Vergleich von Gleitkommazahlen mit Rundungsabweichungen:

```
23 | public static boolean equals(double x, double y) {  
24 |     double eps = 0.000001;  
25 |     return (x == y ||  
26 |           (Math.abs(x - y) /  
27 |            Math.max(Math.abs(x), Math.abs(y))) < eps);  
28 | }
```

Mathe.java

Beispiel: `Mathe.equals(0.1 + 0.1 + 0.1, 0.3)` liefert `true`

Typumwandlung

In manchen Situationen wird ein Wert eines Typs **automatisch** in einen Wert eines anderen Typs **umgewandelt**

Zahlen: bei **Operation auf Ganzzahl- und Gleitkommawert** wird

- 1 der Ganzzahl- in einen Gleitkommawert umgewandelt
- 2 die Operation für die beiden Gleitkommawerte ausgewertet

Beispiel: $13 / 5.0$ wird umgewandelt in $13.0 / 5.0$ mit Wert 2.6
(im Gegensatz zu $13 / 5$ mit Wert 2)

Typumwandlung zu String

Zeichenketten: bei Auswertung von $s + w$ mit einer Zeichenkette s und einem Wert w von anderem Typ wird

- 1 w in eine Zeichenkette umgewandelt
- 2 diese Zeichenkette an s angehängt

Beispiel:

```
int i = 17;  
String s = "Trick " + i;    // weist "Trick 17" zu
```

Hinweis: `" " + w` wandelt beliebigen Wert w in Zeichenkette um

Reihenfolge der Auswertung

Auswertung von Operationen geschieht in bestimmter Reihenfolge, die für alle Operatoren festgelegt ist – entspricht meist Intuition:

$3 * 7 + 5$ wie $(3 * 7) + 5$ („Punkt- vor Strichrechnung“)

für $+$ (wie meiste Operatoren): von links nach rechts

$4 + 3 + \text{"x"}$ $\rightarrow$ "7x" (erst Summe, dann Umwandlung)

$\text{"x"} + 4 + 3$ $\rightarrow$ "x43" (jeweils Umwandlung)

$\text{"x"} + (4 + 3)$ $\rightarrow$ "x7" (erst Summe, dann Umwandlung)

Klammerung erlaubt allgemein, die Reihenfolge zu ändern

Beispiel: Typen, Ausdrücke, Umwandlung

```
1 public class Geometrie {  
2     public static double umfangREck(double breite,double hoehe){  
3         return 2 * (breite + hoehe);  
4     }  
5 }
```

```
1 public class GeometrieTest {  
2     public static void main(String[] args) {  
3         double b = 3;  
4         double h = 5;  
5         double u = Geometrie.umfangREck(b, h);  
6         System.out.println("Rechteck " + b + " x " + h + ": "  
7                             + "Umfang " + u + " [" +  
8                             (Mathe.equals(u, 16) ? "OK" : "falsch")  
9                             + "]");  
10    }  
11 }
```

Übung: Zeit.dauer

Aufgabenblatt 03, Aufgabe 02